THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

OBJECT ORIENTED PROGRAMMING

LAB Assignment 2

SUBMITTED BY:

Name: Rohan

Roll no: 1024150187

Batch: 2O23

Question1: Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members (properties):

- Name
- Roll No
- Degree
- Hostel
- Current CGPA

Member Function (behavior):

- addDetails();
- updateDetails();
- updateCGPA();
- updateHostel();
- displaydetails();

INPUT

```
#include <iostream>
using namespace std;
struct Student {
   string name;
   int rollNo;
   string degree;
   string hostel;
   float cgpa;
   void addDetails() {
      cout << "Enter Name: ";
      cin >> name;
      cout << "Enter Roll No: ";
      cin >> rollNo;
      cout << "Enter Degree: ";
      cin >> degree;
      cout << "Enter Hostel: ";
      cin >> hostel;
      cout << "Enter CGPA: ";
      cin >> cgpa;
   }
   void updateDetails() {
      cout << "Enter new Degree and Hostel:
";
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/MinGW/b
Enter Roll No: 208
Enter Degree: btech
Enter Hostel: q
Enter CGPA: 9
Enter updated CGPA: 8.7

Student Details
Name: pranjal
Roll No: 208
Degree: btech
Hostel: q
CGPA: 8.7
PS C:\Users\pranj\Desktop\oops>
```

```
        cin >> degree >> hostel;
    }
    void updateCGPA() {
        cout << "Enter updated CGPA: ";
        cin >> cgpa;
    }
    void updateHostel() {
        cout << "Enter new Hostel: ";
        cin >> hostel;
    }
    void displayDetails() {
        cout << "\nStudent Details\n";
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Degree: " << degree << endl;
        cout << "Hostel: " << hostel << endl;
        cout << "CGPA: " << cgpa << endl;
    }
};
int main() {
    Student s;
    s.addDetails();
    s.updateCGPA();
    s.displayDetails();
    return 0;
}
```

Question2: Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public scope.

ANSWER2: In C++, public access specifier allows class members to be accessed from outside the class, making them suitable for defining interfaces. On the other hand, private access specifier restricts access to class members only within the class, ensuring data hiding and security. By using private data members and public member functions, encapsulation is achieved.

INPUT

```
#include <iostream>
using namespace std;
class Student {
private:
    float cgpa;
public:
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/Mi
Name: Pranjal
Roll No: 101
CGPA: 8.9
PS C:\Users\pranj\Desktop\oops>
```

```
    string name;
    int rollNo;
    void setCGPA(float c) {
        cgpa = c;

    }
    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "CGPA: " << cgpa << endl;
    }
};
int main() {
    Student s;
    s.name = "Pranjal";
    s.rollNo = 101;
    s.setCGPA(8.9);
    s.display();
    return 0;
}
```

QUESTION3: Create a code snippet that illustrates the following:
Calling of private member functions inside public member function.

INPUT

```
#include <iostream>
using namespace std;
class Demo {
private:
    void privateFunc() {
        cout << "Private Function Called" <<
endl;
    }
public:
    void publicFunc() {
        privateFunc();
    }
};
int main() {
    Demo d;
    d.publicFunc();
    return 0;
}
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/Mir
Private Function Called
PS C:\Users\pranj\Desktop\oops>
```

QUESTION4: Define a class Rectangle with variables width and height of integer type along with following:

(a) void getdata() to initialize object values

(b) void calculatearea() to calculate and display the area.

INPUT

```
#include <iostream>
using namespace std;

class Rectangle {
   int width, height;

public:
   void getdata() {
      cout << "Enter width and height: ";
      cin >> width >> height;
   }

   void calculatearea() {
      cout << "Area = " << width * height <<
endl;
   }
};

int main() {
   Rectangle r;
   r.getdata();
   r.calculatearea();
   return 0;
}
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/Mi
Enter width and height: 10 5
Area = 50
PS C:\Users\pranj\Desktop\oops>
```

QUESTION5: Define a class Complex with variables real and imaginary along with following:

(a) void setComplex(float, float) to initialize object values.

(b) void displayComplex() to show the complex number

(c) Pass and return objects to calculate sum of two complex numbers. Display the sum.

INPUT

```
#include <iostream>
using namespace std;
class Complex {
   float real, imag;
public:
   void setComplex(float r, float i) {
      real = r;
      imag = i;
   }
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/Mir
Sum = 4 + 6i
PS C:\Users\pranj\Desktop\oops>
```

```
    void displayComplex() {
        cout << real << " + " << imag << "i" <<
endl;
    }
    Complex add(Complex c) {
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }
};
int main() {
    Complex c1, c2, c3;
    c1.setComplex(2.5,  3.5);
    c2.setComplex(1.5, 2.5);
    c3 = c1.add(c2);
    cout << "Sum = ";
    c3.displayComplex();
    return 0;
}
```

QUESTION6: Implement scope resolution operator :: for the following uses:

(a) Class functions defined outside the class

INPUT

```
#include <iostream>
using namespace std;
class Test {
public:
    void show();
};
void Test::show() {
    cout << "Function defined outside class"
<< endl;
}
int main() {
    Test t;
    t.show();
    return 0;
}
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:
Function defined outside class
PS C:\Users\pranj\Desktop\oops>
```

(b) Access a global variable with same name as a local variable

INPUT	OUTPUT

```
#include <iostream>
using namespace std;
int x = 10;
int main() {
   int x = 20;
   cout << "Local x = " << x << endl;
   cout << "Global x = " << ::x << endl;
   return 0;
}
```

```
PS C:\Users\pranj\Desktop\oops> C:/Mi
Local x = 20
Global x = 10
PS C:\Users\pranj\Desktop\oops>
```

(c) Access static variables

INPUT

```
#include <iostream>
using namespace std;
class Demo {
public:
   static int count;
};
int Demo::count = 5;
int main() {
   cout << Demo::count;
   return 0;
}
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/
5
PS C:\Users\pranj\Desktop\oops>
```

(d) Use inbuilt libraries (cin cout with scope resolution operator)

INPUT

```
#include <iostream>
int main() {
   std::cout << "Using scope resolution
operator" << std::endl;
   return 0;
}
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/Mi
Using scope resolution operator
PS C:\Users\pranj\Desktop\oops>
```

QUESTION7: Create a code to implement the namespace and use similar variables and functions defined in different code sections.

INPUT

```
#include <iostream>
using namespace std;
namespace First {
   int x = 10;
   void show() {
      cout << "First namespace x = " << x <<
endl;
   }
```

OUTPUT

```
PS C:\Users\pranj\Desktop\oops> C:/MinGW
First namespace x = 10
Second namespace x = 20
PS C:\Users\pranj\Desktop\oops>
```

```
}
namespace Second {
    int x = 20;
    void show() {
        cout << "Second namespace x = " << x
<< endl;
    }
}
int main() {
    First::show();
    Second::show();
    return 0;
}
```